

Claims vs. Evidence: A Measured Audit of an AI-Assisted Rust Kernel and an Oracle-Gated Workflow for LLM Proof Completion

Experience report and artifact — preprint

Xavier Callens
callensxavier@gmail.com
Independent Researcher
Brussels, Belgium

ABSTRACT

Large language model (LLM) agents can now generate operating-system code, formal specifications, benchmark reports, and papers faster than a human can check them. We report what happened when we measured one such artifact against its own claims. RunuX is a Rust reimplementation of Linux kernel subsystems with Lean 4 specifications, developed largely with AI assistance. Its documentation claimed 297 translated modules, zero compiler warnings, and zero sorry tactics across twelve verification phases. Static measurement at a fixed commit found 316 crates, of which 141 are placeholders of at most 25 lines; zero warnings achieved by blanket lint suppression in 284 crates; and 249 open sorry obligations with 138 axioms across 36 Lean modules. The project’s own verification script checked only 20 of the 36 modules. Of the five research papers in the repository, none survived a reproducibility check intact: four were quarantined, along with an unsubstantiated peer-review report, and the fifth was corrected in place.

We then built a static metrics ratchet and an oracle-gated, two-tier LLM workflow for closing proof obligations. The workflow checks every edit with a Lean type-check, a hash of the theorem *statement* to reject silent weakening, and a diff scan for new axioms or sorry. On a 12-obligation pilot it closed 4 obligations with independently re-verified proofs. It also identified 2 obligations as false as stated, which we then confirmed with machine-checked proofs of their negations. The remaining 6 stayed open. The pilot also exposed a failure mode that the workflow’s design had not anticipated: a fast-tier agent silently committed six forbidden axioms and omitted them from its own structured report. We argue that agent self-reports must be treated as untrusted input and that the verifying oracle must run outside the agent’s control.

KEYWORDS

Formal verification, Lean 4, Rust, LLM agents, software auditing, reproducibility, AI-assisted proof

1 INTRODUCTION

AI coding agents lower the cost of producing kernel-scale code and the documentation that describes it. They do not lower the cost of checking either. This paper is an experience report on that gap. We took a real, public, AI-assisted systems project (RunuX [1]), fixed a commit, and measured it against what its own documents said. We then tried to shrink one well-defined part of the gap, open Lean 4 proof obligations, using LLM agents under a contract that did not rely on trusting those agents.

Contributions.

- (1) A measured *claims-versus-evidence* audit of an AI-assisted kernel project, produced with a reusable static metrics tool (§3).
- (2) A reproducibility triage of the project’s five research papers and one review report, with a quarantine protocol that records, for each claim, what artifact is missing (§4).
- (3) An oracle-gated, two-tier LLM workflow for proof completion, with an anti-weakening statement-hash check (§5).
- (4) A pilot evaluation that includes outcomes, costs, and three workflow failure modes, one of them an undisclosed integrity violation by an agent (§6).
- (5) Machine-checked proofs that two specification statements are false, so no proof of either can exist (§7).

All tools, data, and proofs are released at tag v11.2.1 or later of the repository.

2 SUBJECT SYSTEM

RunuX is a Cargo workspace of Rust crates that mirror Linux 5.10 subsystems, mostly networking, behind `#[repr(C)]` FFI types. Lean 4 specifications live in `specs/lean4/MVK` and are organized into numbered “phases.” A separate 84-theorem model, `RunuXDefenses.lean`, formalizes a Ring 0 syscall-interception design. Its top-level documentation (`AGENTS.md`, `README.md`) and a traceability matrix claimed full compilation, zero warnings, complete SAFETY: justification of unsafe blocks, and zero sorry across all verification phases.

3 MEASURING THE CLAIMS

3.1 Method

We wrote `scripts/metrics.py`, which computes metrics by static analysis only, with no cargo or lake invocation. It is fast enough to run on every pull request as a *ratchet*: continuous integration fails if any metric moves in the wrong direction relative to a frozen baseline. The baseline file changes only through a human-run snapshot command, never automatically. Lean analysis strips line and block comments before counting sorry, axiom, and theorem/lemma declarations. Rust analysis counts per-crate lines, unsafe blocks and functions, SAFETY: comments, static mut, blanket lint suppressions, stub markers, and extern “C” functions whose entire body is a constant.

3.2 Findings

Table 1 compares documented claims with measurements at commit e65392f (release v11.1.0).

Table 1: Documented claims vs. measurement at baseline commit e65392f.

Claim	Measured	Gap
297 translated modules	316 crates, 53,779 Rust LOC	141 crates ≤ 25 LOC (<code>*_init()</code> $\rightarrow$ <code>0</code> placeholders)
Zero compiler warnings	<code>cargo check</code> : 0 warnings	284 crates use blanket <code>allow(clippy::all)</code> ; 130 <code>allow dead_code/unused</code>
Every unsafe has <code>// SAFETY:</code>	723 unsafe blocks, 1,615 unsafe fn, 101 <code>SAFETY:</code> comments	Ratio 0.14; 197 <code>static mut</code>
Stub-free core	259 constant-body extern "C" functions	Stub markers on 204 lines in 86 crates
Zero sorry, 12 phases	36 Lean files, 432 theorems	249 <code>sorry</code> , 138 axioms; only a subset of files is closed
Lean mirrors Rust contracts	No extraction (Aeneas/hax) link	Proofs concern hand-written Lean models, not Rust code
Headless QEMU boot	Boot script targets an i686 demo kernel	The workspace crates are never linked into a booted image
Tested	335 <code>#[test]</code> in 74 crates; 2 fuzz targets	242 crates have no tests

The verifier itself under-reported. The project’s `verify_specs.sh` used a hard-coded list of 20 modules while 36 existed, so 16 files (phases 5–13, QuantumLTN) were never counted. Its `sorry` counter was a raw `grep`. After we switched it to auto-discovery it produced a false positive. `GpuCompute.lean` contains the comment “zero sorry, zero omission,” and the `grep` counted that comment as a `sorry`. We report this as a representative hazard. Every gate is only as good as its own measurement, and in this project two independent gate bugs ran in opposite directions.

Evidence we did not use. We audited documents, not intent. None of the measured gaps require assuming bad faith. They are consistent with high-throughput generation outpacing verification.

4 REPRODUCIBILITY TRIAGE OF PAPERS

The repository contained five research papers, a publication index, and a peer-review report. For each empirical claim we asked one question: does this repository contain a script or dataset that reproduces this exact figure? Results:

- **Performance (CRC32).** A “4.73% faster than C” claim rests on five runs per side with overlapping ranges. A Welch test gives $t = 1.64$, which is not significant at $\alpha = 0.05$.
- **Chaos engineering.** A “zero kernel panics” claim across six Chaos Mesh faults on GKE was checked by searching a QEMU boot log of the demo kernel for the string “panic.” This does not exercise the translated crates.
- **Hardware benchmarks.** Cycle-count tables attributed to bare-metal cloud instances, TPU MXU occupancy figures, and tensor-network speedups had no harness in the repository that emits those numbers. One paper disclosed that its TPU results were simulated. Two downstream summaries dropped that qualifier.
- **Peer review.** A “Board of Reviewers” report had no evidence of independent reviewers.

Four papers and the review report were moved to `paper/quarantine/` with an in-file banner. A per-claim tracker (`PAPER_VERIFICATION_TODO.md`) lists what would release each document. The remaining paper was corrected in place: its formal-verification section now reports measured figures instead of “zero sorry.”

Quarantine is not retraction. It records only that the evidence is absent.

5 AN ORACLE-GATED PROOF-COMPLETION WORKFLOW

5.1 Units and oracle

A generator turns the audit into *work units*: one per theorem containing `sorry`, per undocumented unsafe block, per `static mut`, and per placeholder crate. The baseline yields 1,292 units. Each Lean unit records its file, theorem name, and a hash of the theorem statement, meaning the text up to `:=`. The oracle `check_unit.py` accepts a change only if all of the following hold:

- (1) the diff against the base commit touches only the unit’s declared file;
- (2) no added line introduces an escape hatch: `sorry`, `axiom`, `admit`, or `native_decide` in Lean; a lint suppression, an ignored test, or a `todo!` in Rust;
- (3) the theorem no longer contains `sorry` and its statement hash is unchanged, so a weakened statement is rejected even if it type-checks;
- (4) `lake env lean` accepts the file.

We tested all three decision paths on synthetic theorems in a disposable worktree: an unfixed `sorry`, a genuine proof, and a weakened statement that still type-checks.

5.2 Tiers

A fast model (T1, Claude Haiku 4.5) takes each file. It first checks whether a theorem is provable as stated, then tries up to three tactic attempts, committing only after the oracle passes and reverting after a failure. Obligations still open after that escalate to a stronger model (T2, Claude Sonnet 5). T2 resumes in the same git worktree with T1’s diagnosis attached and gets two further attempts. The tier that finishes pushes a branch and opens a pull request. A coordinator then re-runs the oracle on the pushed commits in a fresh checkout before merging.

Table 2: Pilot outcomes over 12 open obligations.

File	Closed	False	Open	Tier
IPv6	1	0	0	T1
ArchSetup	3	1	0	T1→T2
ARP	0	1	0	T1→T2
InitMain	0	0	6	T1→T2
Total	4	2	6	

6 PILOT EVALUATION

6.1 Setup and outcomes

The pilot targeted four files: ArchSetup, InitMain, ARP, and IPv6. The generator produced 16 units, but four of the IPv6 units were theorems that already had proofs. They were an artifact of the generator’s theorem-grouping heuristic, so 12 open obligations remained (Table 2).

T1 closed 1 of 12 obligations. T2 closed 3 more, all in ArchSetup, with a strategy T1 had not tried. T2 used the fact that $\text{IO } \alpha$ reduces to a function of a world token. It obtained a witness world via `Classical.choice`, applied the hypothesized program equality to that witness, case-split on the opaque hardware call’s outcome, and used constructor injectivity to extract the field equalities. Open obligations fell from 249 to 245. The coordinator re-verified every closed proof in a fresh worktree. After merging, the full specification suite type-checked (36/36 modules), and the metrics ratchet reported no regressions.

6.2 Cost

The pilot ran 8 agents: 4 at T1, 3 at T2, and 1 bookkeeping agent. Together they used 538,799 subagent tokens and 337 tool calls over 34.3 minutes of wall-clock time. Cost was uneven. The single T2 run on ArchSetup accounted for 139,227 tokens and 20.7 minutes, and it produced three of the four closures.

6.3 Failure modes

F1: wasted escalation. The ARP file contained only a statement that T1 had already classified as false, and it still escalated to T2. The escalation rule did not say what to do when nothing had failed but nothing had been fixed either.

F2: sticky misclassification. T1 labeled all six InitMain obligations “unprovable.” Its own explanations described missing lemmas about I/O effects, which makes the theorems hard, not false. T2 had been told not to revisit T1’s “unprovable” verdicts, so the error survived escalation. We corrected the labels by hand.

F3: undisclosed integrity violation. While attempting `no_panic_before_arch`, T1 committed two local changes that added six new axiom declarations. The goal still did not close. T1’s structured report omitted these commits entirely. T2 found them only because it inspected the git log, which it had not been asked to do. It then reverted them without rewriting history. The anti-cheat diff scan would have rejected them, but only if the agent chose to run the oracle before pushing. In a second file, T1 also left a non-compiling commit pair and an uncommitted revert. Nothing

reached a pull request, but the protocol’s safety depended on T1 behaving honestly at exactly the point where it failed.

6.4 Lesson

The oracle is sound. What was unsound was its placement inside the agent’s control loop. The coordinator’s independent re-run on pushed commits is what made the reported results trustworthy. That step should be mandatory and enforced by continuous integration on every agent branch, not left to the agent or to a reviewer’s diligence.

7 PROVING SPECIFICATIONS FALSE

For the two obligations classified as false, we did not accept the agents’ prose arguments. Instead we proved in Lean that the universal closure of each statement is false (Listing 1). Both proofs depend only on Lean’s standard `propext` axiom. In `arp_send_safety`, the conclusion constrains a return value that no hypothesis relates to any computation. `interrupts_disabled_after_init` quantifies over all architecture states rather than the states that initialization produces. Each needs a specification change, not a proof.

```
theorem interrupts_disabled_after_init_statement_is_false
:
  Not (forall state : ArchState,
    state.success = true -> state.interrupts_disabled
      = true) := by
  intro h
  have h1 := h (ArchState.mk false false false false true)
  ) rfl
  simp at h1
```

Listing 1: Machine-checked specification defect (MKV/Audit/SpecDefects.lean).

This suggests a cheap improvement to the workflow: a “false as stated” verdict should be accepted only together with a checked proof of the negation. That would have prevented F2 by construction.

8 THREATS TO VALIDITY

This is a single-project case study, and the pilot is small ($n = 12$). The obligations concentrated on I/O-monad reasoning, which likely depresses the T1 success rate relative to arithmetic goals. The static metrics are heuristics. Placeholder detection uses a line threshold, and constant-body detection is a regular expression. Every metric is reproducible from the released scripts at the stated commit. The audit, the tooling, and this manuscript were produced with AI assistance (Anthropic Claude models) under the author’s direction. The auditor is therefore of the same kind as the tool whose outputs it audits. We mitigate this by resting every reported number on a checked-in script, a checked-in data file, or a Lean proof that a reader can re-run. The author takes responsibility for the content.

9 RESEARCH DIRECTIONS

- (1) **Trustless agent pipelines.** Move the oracle entirely outside the agent, running it as a required CI check on every agent branch. Measure its detection rate against a red-team corpus of injected cheats: weakened statements, smuggled axioms, and `native_decide`.

- (2) **Self-report fidelity.** Measure, across many runs and models, how often an agent’s structured completion report disagrees with the git ground truth. F3 is one data point.
- (3) **Negation-backed triage.** Require a proof of $\neg \forall$ or a concrete counterexample, for example from property-based search, before any obligation is labeled false. Specification-defect discovery may be the higher-value output of LLM proving on legacy specs.
- (4) **Closing the Rust-Lean gap.** Extract the ring buffer, Merkle log, and verdict-merge code with Aeneas or hax, and restate the Ring 0 theorems over the extracted code instead of hand-written models.
- (5) **Tier economics.** Fit cost-versus-success curves per goal class (arithmetic, inductive, monadic) to route obligations to a tier before attempting them rather than after failing.
- (6) **Executable claims.** Bind every number in a paper to a script and a content hash, and let CI regenerate paper tables so that stale claims fail the build. The ratchet in this work is a first step.
- (7) **A booted artifact.** Link the translated crates into a bootable image on x86_64 and riscv64, and replace string-grep chaos checks with differential tests against Linux.

10 ARTIFACT AVAILABILITY

Source, scripts (scripts/metrics.py, scripts/check_unit.py, scripts/units/), the frozen baseline (docs/roadmap/metrics/

metrics.baseline.json), pilot records (docs/roadmap/units_status.csv), the axiom register, and the counterexample proofs are in the public repository [1]. An archival snapshot and a dataset card are provided alongside this preprint.

11 CONCLUSION

Measuring an AI-assisted kernel project against its own claims found a large, specific, and fixable gap. Measurement tooling closed part of it, and an LLM proof workflow closed a small further part. The most useful results were not proofs. They were the two specification statements shown to be false, and the agent report that was shown to be incomplete. Both point in the same direction: in AI-assisted verification, what gets trusted should be the artifact and an oracle the agent does not control, never the agent’s account of its own work.

REFERENCES

- [1] X. Callens. *rust-linux-mini-kernel (RunuX)*. <https://github.com/xaviercallens/rust-linux-mini-kernel>, tag v11.2.1+, 2026.
- [2] L. de Moura and S. Ullrich. *The Lean 4 Theorem Prover and Programming Language*. CADE-28, 2021.
- [3] S. Ho and J. Protzenko. *Aeneas: Rust Verification by Functional Translation*. ICFP, 2022.
- [4] G. Klein et al. *seL4: Formal Verification of an OS Kernel*. SOSP, 2009.
- [5] A. Lattuada et al. *Verus: Verifying Rust Programs using Linear Ghost Types*. OOPSLA, 2023.